FLOWRA · ENGINEERING

Production Operations Playbook

Going-live infrastructure, observability, deployment workflow, and on-call runbook for a multi-tenant Tally analytics SaaS.

Version 1.0
February 2026

Internal study document
Read fully before go-live

FLOWRA — Production Operations Playbook

Version 1.0 · February 2026 · For internal study before go-live

> *The purpose of this document is **not** to make you a DevOps engineer. It is to give you and your team enough shared vocabulary and shared expectations that when something breaks at 11 PM, everyone knows what to do, in what order, and who is doing it.*

---

1 · Executive Summary

FLOWRA is moving from a single-machine development setup to a multi-tenant SaaS serving paying Indian SMEs. Once the first customer pays, three things become true that were not true before:

1. **Downtime has a rupee value.** A 10-minute outage during business hours is approximately ₹2,000–₹5,000 of customer trust evaporating per active tenant.
2. **Your local laptop is no longer the source of truth.** Production is.
3. **You cannot push to "main" and pray.** Every change needs a path that has been tested at least once before reaching customers.

This playbook describes the exact infrastructure, processes, and tools required to make those three truths workable, in increasing order of investment. The bare-minimum monthly cost to operate FLOWRA professionally is **about ₹5,000 / month**. The recommended setup is **₹10,000–₹14,000 / month** and unlocks 24×7 visibility, automated rollbacks, and a sustainable debugging workflow.

You can adopt this in phases — Section 12 lays out a four-week rollout that spreads cost and risk.

---

2 · Why This Matters — The Cost of Bad Workflow

Before any architecture, internalise the two failure modes that kill SaaS companies in their first year:

2.1 Failure Mode A — "It works on my laptop"

You ship a fix. It looks fine on your machine. A customer's screen now shows "₹0" where their cash flow should be. You SSH in to debug, accidentally type `db.users.deleteMany({})` instead of `findMany({})`. Now you have no customers.

Defence: environment isolation (Section 3) and read-replica access for debugging (Section 9.4).

2.2 Failure Mode B — "We don't know it's broken until the customer

calls"

The Tally agent on a customer's machine has been silently failing for three days because of a token expiry edge case. The customer logs in Friday evening, sees outdated numbers, and on Monday tells their CA "this software is unreliable."

Defence: observability (Section 6), heartbeats (Section 8.5), and automated alert rules (Section 6.4).

The rest of this playbook is, fundamentally, a structured way to **avoid these two failure modes** without spending more time on operations than on product development.

---

3 · Environment Topology

You need **three** environments. They are different kinds of things, not different scales of the same thing.

ENVIRONMENT	HOSTED ON	PURPOSE	DATABASE	URL	TOUCHED BY
Local	Developer laptop / Emergent	Active development & feature work	Local MongoDB (Docker)	`localhost:3000`	Developers only
Staging	DigitalOcean Droplet (small)	Pre-release QA, customer-issue reproduction	Atlas M0 (free) cluster	`staging.flowra.in`	Developers + internal QA
Production	DigitalOcean Droplet (medium)	Real customers, real money	Atlas M10 dedicated cluster	`app.flowra.in`	Customers + on-call only

3.1 Why three, not two

Many teams skip staging to save money. They learn the hard way that production is the only honest test of code, and the moment you accept that truth without a staging buffer, every bug becomes a customer-facing bug.

Staging exists for one reason: to be the mirror of production where you can break things on purpose. It runs the same Docker images, the same nginx config, the same TLS, the same env-var injection — only the database and the customer count differ.

3.2 What lives where

```
Local                Staging                Production
─────────            ─────────────          ──────────────────────
docker-compose       docker-compose         docker-compose
    ↓                    ↓                      ↓
React 3000           React 3000 (built)     React 3000 (built)
FastAPI 8001         FastAPI 8001           FastAPI 8001
MongoDB local        Atlas M0 free          Atlas M10 dedicated
.env (committed)     .env (secret)          .env (secret)
```

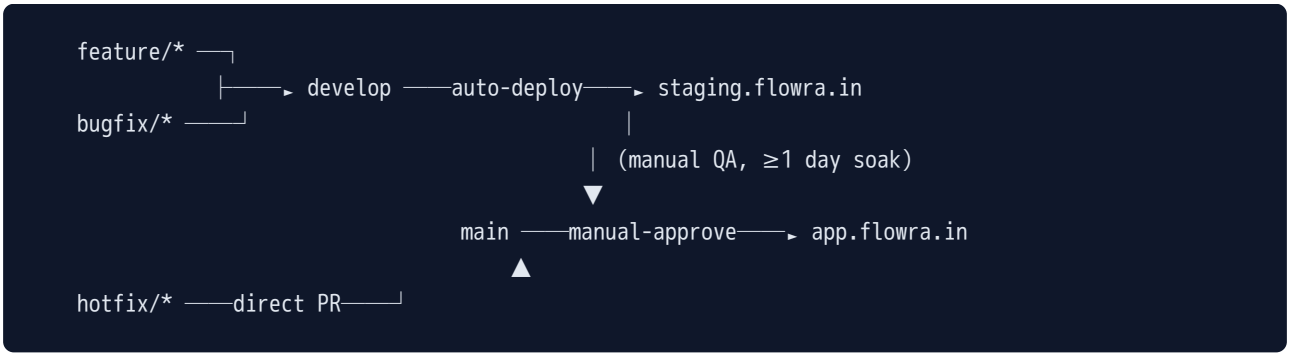
The **only** files that differ between staging and production are:

- `frontend/.env` → `REACT_APP_BACKEND_URL`
- `backend/.env` → `MONGO_URL`, `JWT_SECRET`, `RESEND_API_KEY`
- `nginx.conf` → `server_name` directive

Everything else is byte-identical. This is non-negotiable.

---

4 · Branching & Release Flow



4.1 Rules of the road

BRANCH	PURPOSE	WHO MERGES	REVIEW
`feature/<ticket>`	New work	Developer	1 reviewer
`bugfix/<ticket>`	Non-urgent bugs	Developer	1 reviewer
`develop`	Integration of all in-flight work	Lead dev	Auto-tests must pass
`main`	What is currently running in production	Lead dev only	Manual approval gate
`hotfix/<ticket>`	Production-down emergencies	Anyone with prod access	1 reviewer post-fact

4.2 The "what is in production right now?" problem

There must always be exactly one answer to the question "what code is running in production?" — it is the SHA of the latest commit on `main`. The production deployment includes this SHA in `/api/health` so you can verify remotely:

```
GET /api/health
{ "status": "ok", "version": "9.8.9", "git_sha": "a1b2c3d", "uptime": "4d 12h" }
```

4.3 Hotfix protocol

A **hotfix** bypasses staging only when:

- A paying customer cannot transact, **and**
- Waiting one staging cycle (≥1 day) is unacceptable

Hotfix workflow: 1. Branch from `main`: `git checkout -b hotfix/wrong-cash-flow main` 2. Fix → write a regression test → push 3. PR directly to `main` with manual approval 4. After deploy, **back-merge into** `develop` so the fix is not lost in the next release 5. Post-mortem within 48 hours (Section 11.5)

---

5 · Continuous Integration & Deployment (CI/CD)

GitHub Actions is the recommended runner. Three workflow files live in `.github/workflows/`:

5.1 `test.yml` — runs on every Pull Request

Triggers on every push to a feature/bugfix branch. Blocks merges if any step fails.

What it runs: - Backend: `pytest backend/tests/` (your existing iteration_60 → iteration_93 suite) - Backend lint: `ruff check backend/` - Frontend: `cd frontend && yarn install --frozen-lockfile && yarn build` - Frontend lint: `cd frontend && yarn lint` - (Optional) Lighthouse audit on the built frontend

Time budget: 4–6 minutes total. If it grows past 10 minutes, parallelise the backend test matrix.

5.2 `deploy-staging.yml` — runs on merge to `develop`

Auto-deploys to staging. Steps:

1. SSH into the staging droplet (using a deploy key stored in GitHub Secrets)
2. `cd /opt/flowra && git pull origin develop`
3. `docker compose pull && docker compose up -d --build`
4. Wait 15 seconds, hit `/api/health`. If non-200, automatic rollback to the previous git SHA via `git reset --hard HEAD~1 && docker compose up -d`
5. Slack/email a summary: "Staging is now on commit `a1b2c3d` (subject: …)"

5.3 `deploy-production.yml` — manual trigger from `main`

This is the only deployment path that touches paying customers. Two guardrails:

1. **GitHub Environment protection rule** — requires explicit "Approve & deploy" click from a designated reviewer (you, initially)
2. **Blue-green deploy** — start the new container while the old one is still serving traffic, switch nginx upstream, drain the old container over 30 seconds. If health checks fail at any step, abort and retain the old container.

Manual trigger is intentional friction. It forces you to consciously decide "yes, this code goes to customers now."

5.4 Secrets management

GitHub repository **Secrets** store: - `STAGING_SSH_KEY`, `PROD_SSH_KEY` — SSH private keys for the droplets - `MONGO_URL_PROD`, `MONGO_URL_STAGING` — Atlas SRV connection strings - `JWT_SECRET_PROD`, `JWT_SECRET_STAGING` - `RESEND_API_KEY`, `EMERGENT_LLM_KEY` - `SENTRY_AUTH_TOKEN`

These are **never** in a file checked into git. The CI job writes them into the droplet's `.env` files at deploy time, then the docker container reads them via `env_file:`.

---

6 · Observability — Knowing Before The Customer Calls

This is the highest-leverage spend in operations. ₹3,500 / month of tooling saves you 20+ hours of debugging per month and saves customer relationships worth orders of magnitude more.

6.1 The four pillars

PILLAR	QUESTION IT ANSWERS	TOOL
Errors	What broke?	Sentry
Logs	Why did it break? What was the user doing?	BetterStack
Metrics	Is the system healthy right now?	Atlas + UptimeRobot + droplet `htop`
Traces	Why is it slow?	Sentry Performance (free tier)

6.2 Sentry — set up first

Sentry catches exceptions in both backend (Python SDK) and frontend (React SDK), groups them by stack trace, and tells you:

- The customer's `tenant_id` (we add it as a tag on every event)
- The last 10 actions the user took (breadcrumbs)
- The exact request body, query string, headers
- How many other customers hit the same error

Setup: - Backend: `pip install sentry-sdk[fastapi]`, init in `server.py` with `traces_sample_rate=0.1` (sample 10% of requests for performance traces) - Frontend: `yarn add @sentry/react`, init in `App.js`. Wrap your error boundaries. - Filter out: 401/403 (those are user errors, not bugs), 404 on static assets

Alert rules to configure on day one: - Any new error type fires a Slack/email alert immediately - Same error fires >50 times in 1 hour → escalate to phone - Error rate > 1% of requests for any 5-minute window → page on-call

6.3 Logs — BetterStack or self-hosted Loki

Sentry tells you what crashed. Logs tell you the surrounding context. Every backend request and every Tally agent sync emits structured JSON logs:

```
{"ts":"2026-02-15T14:30:01Z","level":"INFO","tenant_id":"3079...","route":"/api/sales","duration_ms":42,"status":200}
```

Why structured? So you can answer "show me all requests by tenant `3079...` between 14:25 and 14:35 that returned 5xx" in one query, in seconds, without tailing a 2-GB log file.

Recommended: BetterStack (₹1,500/mo for 30-day retention) — drop-in, hosted, has alerting built in. Self-hosting Loki saves money but costs you 8–12 hours of setup.

6.4 Uptime — UptimeRobot (free)

External pings every 60 seconds: - `https://app.flowra.in/api/health` (backend) - `https://app.flowra.in/` (frontend) - `https://staging.flowra.in/api/health` (staging — separate alerts)

When down: SMS + email + Slack. Set the public status page so customers can self-check before contacting you.

6.5 Database — Atlas built-in

MongoDB Atlas dashboards already give you: - Slow query log - Connection pool saturation - Disk usage trend - Replication lag

Configure alerts for: - Connections > 80% of pool size - Slow queries > 1 second - Disk usage > 80%

---

7 · Database Operations (Atlas)

7.1 Cluster sizing

TIER	RAM	STORAGE	COST	USE
M0	shared	512 MB	Free	Staging only
M2	shared	2 GB	~₹720/mo	First 10 customers, validation phase
M10	2 GB	10 GB	**~₹4,500/mo**	**Recommended for paid customers**
M20	4 GB	20 GB	~₹12,000/mo	Once you cross ~50 paying customers

M10 is the sweet spot to start. It includes: - Daily snapshots auto-retained 7 days - Continuous backup (point-in-time restore to any moment in last 24 hours) - Performance Advisor (recommends indexes) - Multi-AZ replication

7.2 Backup & restore drill (do this once a month)

A backup that has never been restored is a wish, not a backup. Calendar a monthly drill:

1. Pick a random tenant from production
2. Restore yesterday's snapshot to the staging Atlas cluster
3. Log in as that tenant on `staging.flowra.in`
4. Verify dashboard, sales, ledgers all render with yesterday's data
5. Document any restore-time issues in `/app/docs/RESTORE_DRILL_LOG.md`

7.3 Migrations

Schema changes go through `backend/migrations/<timestamp>_<name>.py`, each idempotent (safe to re-run):

```
# backend/migrations/2026_03_01_add_dispatch_invoice_changed_index.py
async def up(db):
    await db.dispatch_cards.create_index("invoice_changed", background=True)
async def down(db):
    await db.dispatch_cards.drop_index("invoice_changed_1")
```

A startup hook in `server.py` checks `db.migrations_log` and runs any unapplied scripts. **Never run migrations from your laptop against production.** They run via a separate CI job tied to a tagged release.

7.4 Index audit

Once a quarter, review Atlas Performance Advisor and the slow query log. Drop unused indexes (they hurt write throughput) and add suggested ones that are hit frequently.

7.5 Read-only debug user

Create an Atlas user `flowra_readonly` with `readAnyDatabase` privilege. Use this user (not the production app user) for ad-hoc debugging queries. You **cannot** accidentally `deleteMany` with read-only credentials.

---

8 · Tally Agent Updates — The Hard One

The Tally agent is unique because it runs on **customer-owned Windows machines** that you do not have SSH access to. Every update is shipped across the internet to maybe-flaky home/office connections behind unknown firewalls.

8.1 Versioning rules

- **Patch (`v9.8.x`)** — bug fixes only, fully backwards-compatible. Auto-update.
- **Minor (`v9.x.0`)** — new features, backwards-compatible API. Soft-prompt update.
- **Major (`v10.0.0`)** — breaking changes (new auth flow, new endpoints). Force-update with a 30-day grace window communicated by email.

8.2 Distribution channel

The .exe is hosted at:

```
https://app.flowra.in/agents/FlowraTallyAgent_v9.8.9.exe          (latest stable)
https://app.flowra.in/api/agent/latest-version                    (metadata)
```

The metadata endpoint returns:

```
{
  "version": "9.8.9",
  "url": "https://app.flowra.in/agents/FlowraTallyAgent_v9.8.9.exe",
  "sha256": "a1b2c3...",
  "min_required": "9.8.5",
  "release_notes_url": "https://app.flowra.in/release-notes/9.8.9"
}
```

8.3 Auto-update flow inside the agent

Every 6 hours the GUI hits `/api/agent/latest-version`:

1. If `current < min_required` → modal "Update Required", forced
2. If `current < latest` → toast "v9.9.0 available — Install now / Later"
3. User clicks Install → download .exe to `%TEMP%`, verify sha256, write a small `update.bat` that: - Waits 3 seconds for the current process to exit - Replaces the old .exe with the new one - Re-launches the new .exe
4. The bat is self-deleting

Code-signing is mandatory for this — Windows blocks unsigned silent binary replacement after the first manual run. Until you buy a cert (₹3-5k/yr), users will see a SmartScreen warning on every update.

8.4 Rollback channel

If a release breaks customers, change `/api/agent/latest-version` to point back at the previous .exe. Agents will downgrade themselves on their next 6-hour check. Keep the previous 3 versions in `frontend/public/agents/`.

8.5 Heartbeat & sync telemetry

Every successful sync POSTs to `/api/agent/sync-progress` (already exists). Add a passive heartbeat:

- Agent emits `POST /api/agent/heartbeat` every 5 minutes with `{tenant_id, version, last_sync_at, tally_reachable}`
- Backend stores the latest in `agent_heartbeats`
- Customer Health tab shows agent status:
- 🟢 Heartbeat in last 10 min
- 🟡 Last 30 min
- 🔴 Stale > 30 min — proactively reach out before customer notices

9 · Production Runbook — Debugging Customer Issues

When a customer reports a bug, follow this exact sequence. Do not skip steps "to save time" — the steps exist because each one has saved 30+ minutes of debugging in the past.

9.1 Triage — first 5 minutes

1. Acknowledge to the customer ("Looking at this now, will respond in 30 min")
2. Get their `tenant_id` — find their email in Atlas → users → copy `tenant_id`
3. Open Sentry, filter `tenant_id:<value>`, time range last 24 hours
4. Open BetterStack, same filter
5. **80% of bugs are visible on these two screens immediately.** If you find the error in the first 5 minutes, jump to step 9.4.

9.2 Reproduce — next 30 minutes

If Sentry doesn't have it (the bug doesn't throw an exception, just produces wrong numbers):

1. Atlas → restore the customer's collections to staging
2. `staging.flowra.in` → log in with a temporary password reset for their account
3. Reproduce the exact action they described
4. Watch the staging logs in real-time (`docker compose logs -f backend`)

9.3 Diagnose — capture the root cause

- Add a failing pytest in `backend/tests/test_iteration<N>_<bug>.py` that reproduces the exact bug using the customer's data shape (anonymised)
- Trace through the code with the customer's tenant_id in context
- Form a hypothesis. Validate by running the test.

9.4 Fix — branch + ship

1. `git checkout -b bugfix/wrong-cash-flow`
2. Fix the code
3. The test from step 9.3 must now pass
4. Run the **full** regression suite — `pytest backend/tests/`
5. PR → merge to `develop` → staging deploys automatically
6. Verify on staging using the customer's restored data
7. Merge `develop` → `main` → click "Approve & deploy" in GitHub Actions
8. Verify on production with the customer ("Could you reload and confirm?")

9.5 Post-mortem (only for blocking bugs)

Within 48 hours, write a one-page note in `/app/docs/POST_MORTEMS.md`:

- **What happened** (customer impact, duration)
- **Why it happened** (root cause)
- **Why we didn't catch it sooner** (the interesting one — this finds process gaps)
- **What we changed** (the regression test counts; sometimes also alert rules, runbooks, training)

Post-mortems are **blameless**. They are about systems, not people.

---

10 · Security & Compliance Checklist

10.1 Pre-launch (P0 — must-have)

- □ All secrets out of git history (audit with `git-secrets` or `truffleHog`)
- □ HTTPS via Let's Encrypt on both staging + production
- □ DigitalOcean firewall: only 22, 80, 443 open
- □ SSH key-only login (no password auth), `fail2ban` installed
- □ Atlas user has minimum privileges (no `dbAdmin` for the app user)
- □ Atlas IP allowlist locked to droplet IPs + team IPs
- □ JWT secret rotated from any defaults; ≥256-bit random
- □ Customer passwords hashed with bcrypt (already done)
- □ Rate-limit `/api/auth/login` to 5 attempts / 5 min / IP
- □ CORS locked to your real domains, not `*`
- □ Backups verified (do the restore drill once before launch)

10.2 Within 30 days of launch (P1)

- □ Code-signing certificate for Tally agent
- □ Penetration test by an external firm (₹15-25k one-time)
- □ Customer data export endpoint (DPDP / GDPR readiness)
- □ Customer data deletion endpoint
- □ Privacy policy + Terms of service published
- □ Cyber-liability insurance quote

10.3 Ongoing

- Rotate JWT secret every 6 months (forces re-login of all users)

- Rotate Atlas DB password every 6 months
- Quarterly index audit
- Monthly restore drill
- Annual third-party security review

---

11 · Cost Breakdown

11.1 Bare-minimum P0 — ~₹5,000 / month

ITEM	COST (INR)
DigitalOcean Droplet (production, 2 vCPU / 4 GB)	1,500
MongoDB Atlas M10 cluster	4,500
Domain (annualised)	80
Resend (transactional email)	0 (free tier)
UptimeRobot	0
Sentry	0 (free 5k events)
GitHub	0 (private repo, free for small teams)
Total	**~₹6,080**

11.2 Recommended — ~₹12,000 / month

ITEM	COST (INR)
Above (P0)	6,080
DigitalOcean Droplet (staging, 1 vCPU / 2 GB)	750
BetterStack logs (5 GB / 30-day retention)	1,500
Sentry team plan (50k events)	2,000
Resend (production volume)	1,000
Cloudflare WAF + CDN (Pro plan)	1,700
Total	**~₹13,030**

11.3 One-time costs

ITEM	COST (INR)
Code-signing certificate (1 year)	4,000
Domain (1 year, .in)	800
Penetration test (one-time)	15,000–25,000
Logo / branding refresh (optional)	varies

11.4 Cost-to-revenue sanity check

At a Starter plan of ₹999/month, you need **~6 paying customers** to cover the recommended infra. At your Enterprise plan of ₹3,799/month, **two paying enterprise customers cover everything in 11.2**.

---

12 · Phased Rollout — Four-Week Plan

Week 1 — Containerise & Deploy

- Day 1-2: Write `Dockerfile.backend`, `Dockerfile.frontend`, `docker-compose.prod.yml`
- Day 3: Provision production DO droplet, install Docker, set up `nginx` + `certbot`
- Day 4: Provision Atlas M10, run `mongorestore` of current data
- Day 5: First production deploy. Smoke-test all flows manually.

Week 2 — Staging & CI/CD

- Day 1: Provision staging droplet + Atlas M0
- Day 2-3: Write `.github/workflows/test.yml`, `deploy-staging.yml`, `deploy-production.yml`
- Day 4: Configure GitHub Environment protection rule
- Day 5: Test rollback by intentionally pushing a broken `develop` commit

Week 3 — Observability

- Day 1: Set up Sentry, integrate backend + frontend
- Day 2: Set up BetterStack, route container logs
- Day 3: Configure UptimeRobot + status page
- Day 4: Configure Atlas alerts
- Day 5: Test that an alert actually wakes the right person

Week 4 — Hardening & Documentation

- Day 1-2: Security checklist (Section 10.1) line by line
- Day 3: First restore drill
- Day 4: Write team-specific runbook (this doc, customised with your phone numbers and Slack channels)
- Day 5: Soft-launch to 1-2 friendly customers

After week 4: open the doors.

---

13 · Decisions Matrix — When To Choose What

SITUATION	DECISION
< 5 paying customers	M2 cluster is fine, defer M10
> 10 paying customers OR data > 500 MB	Move to M10 immediately
> 50 paying customers	Add a read replica, consider M20
Hotfix needed during business hours	Use the hotfix workflow (Section 4.3)
Hotfix needed at midnight	Wait 6 hours unless customer is actively losing money
Tally agent regression in production	Roll back the .exe (Section 8.4) before fixing forward
Customer asks for custom feature	Add to backlog; do not branch from `main` for one-off code
You're tempted to debug live in production	Stop. Restore to staging. Debug there.

---

14 · Appendix A — Command Cheatsheet

14.1 Production droplet

```
# SSH in
ssh flowra@app.flowra.in

# See running containers
sudo docker compose -f /opt/flowra/docker-compose.prod.yml ps

# Tail backend logs
sudo docker compose -f /opt/flowra/docker-compose.prod.yml logs -f backend

# Manual deploy (only when CI is broken)
cd /opt/flowra && git pull origin main && sudo docker compose up -d --build

# Roll back
cd /opt/flowra && git reset --hard HEAD~1 && sudo docker compose up -d --build

# Disk usage check
df -h && du -sh /var/lib/docker
```

14.2 Atlas

```
# Connect with read-only user
mongosh "mongodb+srv://flowra-prod.xxxx.mongodb.net/" --apiVersion 1 \
  --username flowra_readonly

# Find a tenant by email
db.users.findOne({ email: "customer@example.in" }, { tenant_id: 1, _id: 0 })

# Top 10 slow queries today
db.system.profile.find({ ts: { $gt: new Date(Date.now() - 86400000) } })
  .sort({ millis: -1 }).limit(10)
```

14.3 Tally agent (on a test Windows machine)

```
REM Force a re-login
FlowraTallyAgent.exe --logout

REM Check version
FlowraTallyAgent.exe --version

REM Register / unregister auto-start
FlowraTallyAgent.exe --register-startup
FlowraTallyAgent.exe --unregister-startup

REM Logs live at:
%LOCALAPPDATA%\Flowra\logs\agent_YYYYMMDD.log
```

---

15 · Appendix B — Decision Worksheet

Print this page. Discuss with your team. Tick a box per row before reading this doc again.

```
Repo & CI
  □ GitHub private repo created
  □ Lead developer is named:  _________________
  □ Approval reviewer for production deploys is named:  _________________

Hosting
  □ DO Droplet for production:  □ 4 GB  □ 8 GB  □ Other ________
  □ DO Droplet for staging:     □ 2 GB  □ skip for now
  □ Atlas tier for production:  □ M10  □ M2  □ skip cloud, stay local
  □ Atlas tier for staging:     □ M0   □ skip

Observability (which to start with)
  □ Sentry        □ now  □ post-launch  □ never
  □ BetterStack   □ now  □ post-launch  □ never
  □ UptimeRobot   □ now  □ post-launch  □ never

Tally Agent
  □ Code-signing certificate budget:  □ approved  □ defer to month 2  □ never
  □ Auto-update channel:              □ build now  □ build after first 10 customers

Phase rollout
  □ We'll do all 4 weeks back-to-back
  □ We'll do Week 1 only, then re-evaluate
  □ Other: _________________________________
```

---

16 · Closing Note

The unglamorous truth of running a SaaS is that operations is a much larger share of your time than feature work, after the first paying customer. This playbook exists so that effort spent on operations is **leveraged effort**: each thing you set up once protects you forever.

When you doubt whether a particular step is "worth it," ask:

> *"How many customer-hours of pain does this prevent over a year?"*

If the answer is more than 1 hour and the cost is less than ₹1,500/month, do it. If the answer is less than 1 hour, defer.

Good luck, and call me when something breaks.

— *FLOWRA Engineering*